การเปรียบเทียบคุณภาพและประสิทธิภาพระหว่างแอพที่พัฒนาด้วยเอไอกับแอพที่พัฒนาโดยมนุษย์

Quality and Productivity Comparison: AI-Generated Apps vs Human-Developed Apps

รชต เทียมโคกกรวด

Rachata Tieamkokkruad

สำนักวิชาศาสตร์ และศิลป์ ดิจิทัล มหาวิทยาลัยเทคโนโลยีสุรนารี

Institute of Digital Arts and Science, Suranaree University of Technology, Thailand.

ABSTRACT

Background and Objectives: The rapid emergence of Generative Artificial Intelligence (AI), specifically Large Language Models (LLMs) like GPT-4, has transformed software development through automatic code generation. However, it remains unclear whether AI can effectively replace human developers in terms of both quality and productivity. This research conducts a systematic comparison between AI-exclusive generation and traditional human development. The objectives are to evaluate productivity via development time and assess software quality based on international standards. Such findings are vital for industries integrating AI into professional workflows.

Methodology: This comparative study focused on a mobile app, "Kin Rai Dee," developed under identical requirements including menu filtering and tournament modes. Gemini 3 Pro was used for the AI group, while a student developer represented the human group. The tech stack included React Native, Expo, and libraries like react-native-paper. Software quality was evaluated via the ISO/IEC 25010 framework, assessing characteristics like Functional Suitability, Reliability, and Maintainability, while productivity was measured through total time and rework cycles.

Main Results: Results reveal a significant productivity gap. AI completed the project in 88 minutes, while the human required 295 minutes, making AI three times faster overall. AI excelled in drafting, taking 48 minutes compared to the human developer's 245 minutes. However, quality assessments revealed that AI was more prone to critical errors during development. The AI approach encountered 6 defects, including 3 high-severity version mismatches and logical errors in tournament pairings for odd-numbered items. The human developer faced only 3 defects. Ultimately, both reached high quality, with AI producing modular code structures comparable to human standards.

Discussions: While AI is exceptionally fast for boilerplate code, it lacks real-time awareness of specific tool versions and complex logical contexts. AI suggested incompatible libraries, requiring human intervention to resolve technical conflicts. This confirms the "Human-in-the-Loop" (HITL) concept; although AI speeds up early production five-fold, the time spent on debugging and maintenance (Phase 2) was nearly equal between AI (40 minutes) and humans (50 minutes). AI is effective for repetitive tasks but struggles with specialized logic where it provides solutions based on patterns rather than actual runtime testing.

Conclusions: This study concludes that AI is a powerful accelerator but not a complete replacement for human expertise. The most efficient model is "Human-in-the-Loop," where AI handles rapid drafting while humans manage verification and logic refinement. To reduce errors, developers should specify tool versions in prompts. Ultimately, maintaining strong fundamental coding skills is essential for developers to manage AI-generated errors and maximize the potential of Generative AI in the software industry.

ARTICLE INFO

Article history:
Received
Received in revised form

Accepted
Available online

Keywords:
Softweare Development
(การพัฒนาซอฟต์แวร์)
Artificial Intelligence
(ปัญญาประดิษฐ์)
AI-Developed
(พัฒนาโดยปัญญาประดิษฐ์)
Human-Developed
(พัฒนาโดยมนุษย์)

*Corresponding author
E-mail address: rachata.gap@gmail.com

บทคัดย่อ

ที่มาและวัตถุประสงค์: การเกิดขึ้นอย่างรวดเร็วของปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative AI) โดยเฉพาะโมเดลภาษาขนาดใหญ่ (LLMs) เช่น GPT-4 ได้เปลี่ยนโฉมหน้าการพัฒนาซอฟต์แวร์ด้วยการสร้างโค้ดอัตโนมัติ อย่างไรก็ตาม ยังไม่มีความชัดเจนว่า AI สามารถทดแทนนักพัฒนาที่เป็นมนุษย์ได้อย่างมีประสิทธิภาพทั้งในด้านคุณภาพและประสิทธิภาพการผลิตหรือไม่ งานวิจัยนี้จึงดำเนินการเปรียบเทียบอย่างเป็นระบบระหว่างการพัฒนาด้วย AI ทั้งหมดกับการพัฒนาโดยมนุษย์แบบดั้งเดิม โดยมีวัตถุประสงค์เพื่อประเมินประสิทธิภาพการผลิตผ่านระยะเวลาที่ใช้ในการพัฒนา และประเมินคุณภาพของซอฟต์แวร์ตามมาตรฐานสากล ซึ่งผลการศึกษาจะมีความสำคัญต่ออุตสาหกรรมที่กำลังนำ AI เข้ามาใช้ในกระบวนการทำงานระดับมืออาชีพ

ระเบียบวิธีวิจัย: การศึกษาเชิงเปรียบเทียบนี้มุ่งเน้นไปที่โมบายแอพพลิเคชั่นชื่อ "Kin Rai Dee" (กินไรดี) ซึ่งพัฒนาภายใต้ข้อกำหนดของระบบชุดเดียวกัน รวมถึงฟังก์ชันการกรองเมนูอาหารและโหมดการแข่งขัน (Tournament mode) กลุ่มที่พัฒนาด้วย AI ใช้ Gemini 3 Pro เป็นเครื่องมือหลัก ในขณะที่กลุ่มที่เป็นมนุษย์คือตัวแทนนักศึกษาผู้พัฒนา เทคโนโลยีที่ใช้ประกอบด้วย React Native, Expo และไลบรารีเสริม เช่น react-native-paper คุณภาพของซอฟต์แวร์ถูกประเมินผ่านกรอบมาตรฐาน ISO/IEC 25010 ในด้านต่างๆ เช่น ความเหมาะสมในการใช้งานตามฟังก์ชัน ความน่าเชื่อถือ และความสามารถในการบำรุงรักษา ในขณะที่ประสิทธิภาพการผลิตวัดจากเวลารวมที่ใช้และรอบการแก้ไขงาน

ผลการวิจัย: ผลการศึกษาแสดงให้เห็นถึงความแตกต่างอย่างมีนัยสำคัญของประสิทธิภาพการผลิต โดย AI สามารถดำเนินโครงงานเสร็จสิ้นภายในเวลาเพียง 88 นาที ในขณะที่มนุษย์ต้องใช้เวลาถึง 295 นาที ซึ่งทำให้ AI มีความรวดเร็วโดยรวมมากกว่าถึง 3 เท่า AI มีความโดดเด่นอย่างมากในช่วงการขึ้นโครงร่างโปรแกรมซึ่งใช้เวลาเพียง 48 นาที เมื่อเทียบกับมนุษย์ที่ใช้เวลา 245 นาที อย่างไรก็ตาม การประเมินคุณภาพพบว่า AI มีแนวโน้มที่จะเกิดข้อผิดพลาดร้ายแรงระหว่างการพัฒนามากกว่า โดยกลุ่ม AI พบข้อบกพร่อง 6 รายการ รวมถึงปัญหาความไม่สอดคล้องของเวอร์ชั่นไลบรารี 3 รายการ และข้อผิดพลาดทางตรรกะในการจับคู่แข่งขันเมื่อจำนวนรายการอาหารเป็นเลขคี่ ในขณะที่มนุษย์พบข้อบกพร่องเพียง 3 รายการ ท้ายที่สุดแล้ว แอพพลิเคชั่นทั้งสองมีคุณภาพอยู่ในเกณฑ์ดี โดย AI สามารถสร้างโครงสร้างโค้ดที่เป็นระเบียบเทียบเท่ากับมาตรฐานของมนุษย์

การอภิปรายผล:แม้ว่า AI จะมีความรวดเร็วอย่างยอดเยี่ยมในการสร้างโค้ดพื้นฐาน (Boilerplate) แต่ยังขาดความตระหนักรู้เกี่ยวกับเวอร์ชั่นของเครื่องมือเฉพาะทางและบริบททางตรรกะที่ซับซ้อนในขณะทำงานจริง AI มักแนะนำไลบรารีที่เข้ากันไม่ได้ ซึ่งต้องใช้นักพัฒนาที่เป็นมนุษย์เข้ามาแก้ไขปัญหาเหล่านั้น สิ่งนี้ยืนยันถึงแนวคิด "Human-in-the-Loop" (HITL) ว่ามีความสำคัญอย่างยิ่ง แม้ว่า AI จะช่วยเร่งการผลิตในช่วงแรกได้ถึง 5 เท่า แต่เวลาที่ใช้ในการแก้บั๊กและบำรุงรักษาของทั้ง AI (40 นาที) และมนุษย์ (50 นาที) นั้นใกล้เคียงกัน AI มีประสิทธิภาพสำหรับงานที่ทำซ้ำๆ แต่ยังคงมีปัญหากับตรรกะเฉพาะทางที่ซับซ้อน ซึ่ง AI มักจะให้คำตอบตามรูปแบบข้อมูลที่เคยเรียนรู้มากกว่าการทดสอบการทำงานจริง

บทสรุป:การศึกษานี้สรุปได้ว่า AI เป็นเครื่องมือเร่งการทำงานที่ทรงพลัง แต่ยังไม่สามารถทดแทนความเชี่ยวชาญของมนุษย์ได้อย่างสมบูรณ์ รูปแบบการทำงานที่มีประสิทธิภาพที่สุดคือ "Human-in-the-Loop" โดยให้ AI จัดการการขึ้นโครงร่างอย่างรวดเร็ว ในขณะที่มนุษย์ทำหน้าที่ตรวจสอบและปรับปรุงตรรกะให้ถูกต้อง เพื่อลดข้อผิดพลาด ผู้พัฒนาควรระบุเวอร์ชั่นของเครื่องมือในพรอมต์ให้ชัดเจน ท้ายที่สุดแล้ว การรักษาทักษะพื้นฐานในการเขียนโค้ดที่แข็งแกร่งยังคงเป็นสิ่งจำเป็นสำหรับนักพัฒนา เพื่อที่จะสามารถจัดการกับข้อผิดพลาดที่ AI สร้างขึ้น และเพิ่มศักยภาพของ Generative AI ในอุตสาหกรรมซอฟต์แวร์ได้อย่างสูงสุด

บทนำ

ปัจจุบันเทคโนโลยีปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative AI) โดยเฉพาะโมเดลภาษาขนาดใหญ่ (Large Language Models: LLMs) ได้เข้ามามีบทบาทสำคัญในการพัฒนาซอฟต์แวร์อย่างกว้างขวาง ซึ่ง Sauvola et al. (2024) ระบุว่าการเปิดตัวโมเดลระดับสูงเช่น GPT-4 ได้เปลี่ยนแปลงกระบวนการทำงานของนักพัฒนาอย่างมีนัยสำคัญ ทั้งในด้านการสร้างโค้ด การออกแบบระบบ และการทดสอบ เทคโนโลยีดังกล่าวช่วยเพิ่มประสิทธิภาพการทำงานได้ถึงร้อยละ 20-50 โดยเฉพาะงานที่ซ้ำซ้อนและการออกแบบหน้าจอผู้ใช้งาน

อย่างไรก็ตาม แม้ว่าปัญญาประดิษฐ์จะช่วยเพิ่มความเร็วในกระบวนการผลิตได้อย่างชัดเจน แต่ยังมีข้อถกเถียงสำคัญเกี่ยวกับคุณภาพ ความถูกต้อง ความสามารถในการบำรุงรักษา (Maintainability) และความปลอดภัยของซอฟต์แวร์เมื่อเทียบกับผลงานที่สร้างโดยมนุษย์ นอกจากนี้ การประเมินคุณภาพงานในโครงการซอฟต์แวร์ยังจำเป็นต้องอาศัยกรอบมาตรฐานสากล เช่น ISO/IEC 25010 หรือมาตรฐานระดับสากลอื่นๆ เพื่อตรวจสอบความครบถ้วนและความถูกต้องของผลลัพธ์

ด้วยเหตุนี้ การวิจัยครั้งนี้จึงมุ่งเน้นการทดลองโดยใช้โจทย์พัฒนาแอพพลิเคชันเดียวกัน เพื่อเปรียบเทียบผลลัพธ์ระหว่างแอพที่สร้างด้วยปัญญาประดิษฐ์และแอพที่พัฒนาโดยมนุษย์อย่างเป็นระบบ เพื่อให้ได้ข้อมูลเชิงประจักษ์ทั้งในด้านประสิทธิภาพการพัฒนา เช่น ระยะเวลาที่ใช้ และด้านคุณภาพซอฟต์แวร์ตามมาตรฐานสากล เพื่อเป็นแนวทางในการตัดสินใจนำปัญญาประดิษฐ์มาใช้ในอุตสาหกรรมซอฟต์แวร์อย่างเหมาะสม

การทบทวนวรรณกรรมและงานวิจัยที่เกี่ยวข้อง

ปัญญาประดิษฐ์เชิงสร้างสรรค์และโมเดลภาษาขนาดใหญ่ (Generative AI&LLMs) ปัญญาประดิษฐ์เชิงสร้างสรรค์เป็นเทคโนโลยีที่สามารถสร้างข้อมูลหรือแนวคิดใหม่ๆ ได้โดยอัตโนมัติจากการเรียนรู้ข้อมูลจำนวนมาก โดยมีโมเดลภาษาขนาดใหญ่ (LLMs) เป็นเทคโนโลยีหลักที่ช่วยให้ AI เข้าใจและสร้างโค้ดที่มีตรรกะใกล้เคียงกับมนุษย์ ในบริบทของการพัฒนาซอฟต์แวร์ LLMs สามารถช่วยแปลงความต้องการให้เป็นโค้ด แนะนำการแก้บั๊ก และสร้างชุดทดสอบอัตโนมัติได้

คุณภาพซอฟต์แวร์และประสิทธิภาพในการพัฒนา คุณภาพของซอฟต์แวร์พิจารณาจากองค์ประกอบหลัก เช่น ความถูกต้อง (Correctness) ความน่าเชื่อถือ (Reliability) และความสามารถในการบำรุงรักษา (Maintainability) ส่วนประสิทธิภาพในการพัฒนาวัดจากความสามารถในการผลิตงานได้รวดเร็วภายใต้ทรัพยากรที่จำกัด เช่น เวลาที่ใช้ และจำนวนข้อผิดพลาดที่พบ

มาตรฐาน ISO/IEC 25010 เป็นมาตรฐานสากลที่ใช้ประเมินคุณภาพซอฟต์แวร์ มี 8 ด้านหลัก ได้แก่ ความเหมาะสมในการใช้งานตามฟังก์ชัน (Functional Suitability) ประสิทธิภาพการทำงาน (Performance Efficiency) ความเข้ากันได้ (Compatibility) ความง่ายต่อการใช้งาน (Usability) ความเสถียร (Reliability) ความปลอดภัย (Security) ความง่ายในการบำรุงรักษา (Maintainability) และความสามารถในการเคลื่อนย้าย (Portability)

งานวิจัยที่เกี่ยวข้อง Sauvola et al. (2024) ได้ศึกษาการเปลี่ยนผ่านของอุตสาหกรรมซอฟต์แวร์เมื่อมีการนำเจนเนอเรทีฟเอไอมาใช้ โดยระบุว่าเอไอสามารถเพิ่มประสิทธิภาพการทำงานได้ร้อยละ 20-50 โดยเฉพาะงานที่ซ้ำซ้อน อย่างไรก็ตาม ยังคงมีข้อจำกัดด้านความถูกต้องและความปลอดภัยซึ่งจำเป็นต้องมีมนุษย์คอยตรวจสอบคุณภาพในรูปแบบฮิวแมนอินเดอะลูปนอกจากนี้ งานวิจัยของไทยยังชี้ให้เห็นว่าการใช้มาตรฐาน เช่น ISO/IEC 29110 ช่วยให้กระบวนการทำงานมีระบบและผลลัพธ์มีคุณภาพดีขึ้นอย่างมีนัยสำคัญ ซึ่งสามารถนำมาปรับใช้ในการประเมินเปรียบเทียบระหว่างงานที่สร้างด้วยเอไอและมนุษย์ได้

วัตถุประสงค์ของการวิจัย

1. เพื่อเปรียบเทียบประสิทธิภาพ (Productivity) ของการพัฒนาแอพพลิเคชันระหว่างการใช้ปัญญาประดิษฐ์เพียงอย่างเดียวในการสร้างแอพและการพัฒนาโดยมนุษย์
2. เพื่อประเมินและเปรียบเทียบคุณภาพ (Quality) ของแอพพลิเคชันที่สร้างด้วยปัญญาประดิษฐ์และแอพพลิเคชันที่พัฒนาโดยมนุษย์ตามมาตรฐานสากล

วิธีการศึกษา

1. รูปแบบการวิจัย การศึกษาครั้งนี้เป็นการวิจัยเชิงเปรียบเทียบ (Comparative Research) โดยมุ่งเน้นการวิเคราะห์ความแตกต่างระหว่างแอปพลิเคชันที่สร้างด้วยปัญญาประดิษฐ์เชิงสร้างสรรค์ (AI-Generative Application) และแอปพลิเคชันที่พัฒนาโดยมนุษย์ (Human-Developed Application) ซึ่งจะทำการเปรียบเทียบองค์ประกอบสำคัญใน 2 มิติ ได้แก่ การเปรียบเทียบคุณภาพซอฟต์แวร์ (Quality) และประสิทธิภาพในการผลิต (Productivity)

2. ขอบเขตการศึกษา ผู้วิจัยกำหนดข้อกำหนดความต้องการ (Requirements) ชุดเดียวกันในการพัฒนาทั้งสองแนวทาง เพื่อควบคุมตัวแปรและสร้างความเท่าเทียมในการเปรียบเทียบมากที่สุด โดยแบ่งการทดลองออกเป็น 2 ส่วน คือ

(1) แอปพลิเคชันที่สร้างโดยปัญญาประดิษฐ์ทั้งหมด และ

(2) แอปพลิเคชันที่พัฒนาโดยมนุษย์จากความรู้ด้านการเขียนโค้ดและการออกแบบระบบ

3. เครื่องมือที่ใช้ในการศึกษา

1. เครื่องมือสำหรับการสร้างซอฟต์แวร์

Gemini 3 Pro สำหรับการสร้างโค้ด ออกแบบส่วนติดต่อผู้ใช้งาน (UI) และวิเคราะห์ข้อผิดพลาดในการพัฒนา AI-Generative Application

Visual Studio Code สำหรับการเขียนและพัฒนาโค้ดซอฟต์แวร์

เฟรมเวิร์ก (Framework) หรือไลบรารี (Library) สำหรับพัฒนาตามข้อกำหนดของระบบ

2. เครื่องมือสำหรับประเมินคุณภาพซอฟต์แวร์ ผู้วิจัยใช้มาตรฐาน ISO/IEC 25010 เป็นกรอบหลักในการประเมินคุณภาพซอฟต์แวร์ ครอบคลุม 8 คุณลักษณะสำคัญ ดังนี้:

Functional Suitability: ความครบถ้วนและความถูกต้องของฟังก์ชัน

Performance Efficiency: ความเร็วและการใช้ทรัพยากรของระบบ

Compatibility: ความสามารถในการทำงานร่วมกับระบบหรืออุปกรณ์อื่น

Usability: ความง่ายต่อการใช้งานสำหรับผู้ใช้

Reliability: ความเสถียรและความถูกต้องในการทำงาน

Security: ความปลอดภัยของข้อมูล

Maintainability: ความง่ายในการแก้ไขและพัฒนาโปรแกรมต่อ

Portability: ความสามารถในการเคลื่อนย้ายไปใช้งานข้ามระบบ

ในการทดลองจริง ผู้วิจัยได้ประยุกต์ใช้การทดสอบฟังก์ชัน (Functional test) การทดสอบประสิทธิภาพ (Load test) การวิเคราะห์ความปลอดภัย (Static analysis) และแบบตรวจสอบความง่ายในการใช้งาน (Usability checklist) เพื่อวัดผลตามคุณลักษณะดังกล่าว

3. เครื่องมือประเมินประสิทธิภาพในการผลิต

ระยะเวลาที่ใช้ในกระบวนการพัฒนา

จำนวนบรรทัดของโค้ดที่สร้างขึ้น

จำนวนข้อผิดพลาด (Bugs) ที่พบระหว่างการพัฒนา

จำนวนรอบที่ต้องแก้ไขงาน (Rework)

4. ขั้นตอนการดำเนินการศึกษา

ขั้นตอนที่ 1 กำหนดความต้องการ: สร้างเอกสารข้อกำหนดความต้องการ (Requirement Document) ในรูปแบบเดียวกันสำหรับการพัฒนาทั้งสองแนวทาง

ขั้นตอนที่ 2 การพัฒนาแอปพลิเคชัน: ใช้อัลกอริทึมของปัญญาประดิษฐ์ในการสร้างส่วนประกอบทั้งหมดของแอปพลิเคชันและบันทึกผล ควบคู่ไปกับการพัฒนาด้วยนักศึกษาผู้พัฒนาซึ่งเขียนโค้ดทั้งหมดตามข้อกำหนดและบันทึกผลทุกขั้นตอน

ขั้นตอนที่ 3 การทดสอบและประเมินผล: ดำเนินการทดสอบตามมาตรฐาน ISO/IEC 25010 และบันทึกข้อมูลเวลา ปริมาณโค้ด ข้อผิดพลาด และจำนวนรอบการแก้ไขเพื่อนำมาวิเคราะห์เปรียบเทียบ

5. กำหนดความต้องการ

ผู้วิจัยกำหนดความต้องการมาตรฐานสำหรับใช้ในการพัฒนาเพื่อให้แนวทางของแอปพลิเคชันทั้งสองชุดตรงกัน ดังนี้:

1. ความต้องการด้านฟังก์ชัน (Functional Requirements):

ระบบคัดกรองเมนูอาหาร: ผู้ใช้เลือกหมวดอาหารได้หลายหมวดพร้อมกัน และมีการตรวจสอบการเลือก

ระบบเลือกโหมดการทำงาน: สามารถเลือกโหมดสุ่มหรือโหมดตัดสินใจได้

ระบบโหมดสุ่ม: สุ่มเมนู 1 รายการ แสดงผลพร้อมข้อมูล และกดสุ่มใหม่ได้

ระบบโหมดตัดสินใจ: ดึงรายการมาจับคู่ทีละคู่เพื่อเลือกผู้ชนะเข้ารอบต่อไปจนถึงเมนูสุดท้าย

การจบการทำงาน: แสดงเมนูสุดท้ายพร้อมปุ่มยืนยันและกลับหน้าหลัก

2. ความต้องการที่ไม่ใช่ฟังก์ชัน (Non-Functional Requirements):

ด้านการใช้งาน: รองรับการใช้งานบนโทรศัพท์ และแสดงผลรวดเร็วชัดเจน

ด้านประสิทธิภาพ: พัฒนาด้วย React Native และประมวลผลจับคู่รวดเร็ว

ด้านข้อมูล: ใช้ Hardcoded Data ในการเก็บข้อมูลอาหารเพื่อเลี่ยงการใช้ Database

3. ไลบรารีเสริมที่กำหนด: มีการกำหนดให้ใช้ไลบรารีมาตรฐาน เช่น react-native-paper, expo-router, react-native-reanimated, และ expo-sharing เพื่อให้ผลลัพธ์ออกมาตรงกันที่สุด

ผลการศึกษา ตัวอักษร Angsana New ขนาด 16 (ตัวอักษรหนา)

จากการดำเนินงานวิจัยเพื่อเปรียบเทียบการพัฒนาแอปพลิเคชันระหว่างปัญญาประดิษฐ์และมนุษย์ ผู้วิจัยได้รวบรวมข้อมูลและวิเคราะห์ผลการศึกษาโดยแบ่งออกเป็นมิติด้านประสิทธิภาพการผลิตและมิติด้านคุณภาพซอฟต์แวร์ ดังนี้

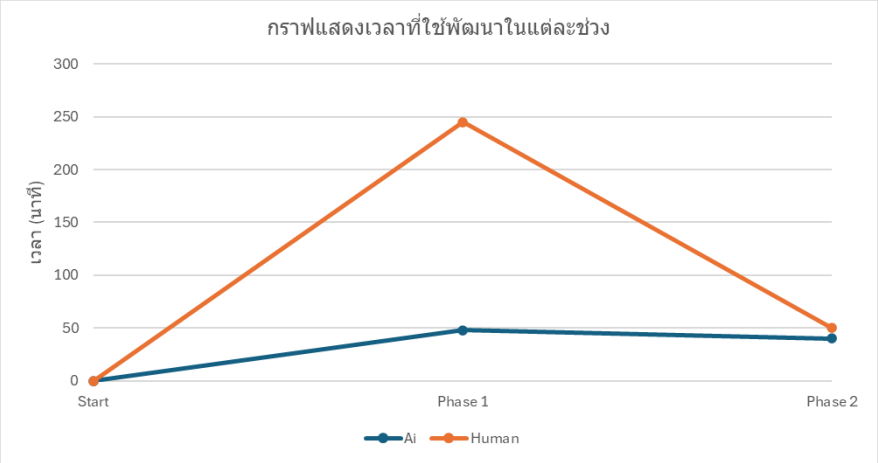
1. ประสิทธิภาพในการพัฒนา ผู้วิจัยบันทึกระยะเวลาที่ใช้ในการพัฒนาแอปพลิเคชัน "กินไรดี" โดยแบ่งออกเป็น 2 ระยะ คือ ระยะที่ 1 การพัฒนาโปรแกรมตามข้อกำหนด (Phase 1) และระยะที่ 2 การตกแต่งพร้อมแก้ไขปัญหา (Phase 2) ซึ่งมีรายละเอียดดังแสดงในตารางที่ 1

ตารางที่ 1 การเปรียบเทียบเวลาที่ใช้ในการพัฒนา (นาที)

Table 1. Table 1. Comparison of Development Time (Minutes)

กลุ่มการพัฒนา (Development Group)	**ระยะที่ 1** (Phase 1)	**ระยะที่ 2** (Phase 2)	**รวมเวลาทั้งหมด** (Total Time)
การพัฒนาด้วยเอไอ (AI-Development)	48	40	88
การพัฒนาโดยมนุษย์ (Human-Development)	245	50	295

จากการศึกษาพบว่าปัญญาประดิษฐ์ มีความรวดเร็วในการพัฒนาโดยรวมมากกว่ามนุษย์ประมาณ 3 เท่า โดยเฉพาะในระยะการขึ้นโครงร่างโปรแกรมที่ใช้เวลาเพียง 48 นาที เมื่อเทียบกับมนุษย์ที่ใช้เวลา 245 นาที อย่างไรก็ตาม ระยะเวลาในการแก้ไขบั๊กและตกแต่งโปรแกรมของทั้งสองกลุ่มมีความใกล้เคียงกัน ดังแสดงในภาพที่ 1



ภาพที่ 1 กราฟแสดงเวลาที่ใช้พัฒนาในแต่ละช่วง

Figure 1. Comparison of Software Development Time between AI and Human Developers

2. ผลด้านคุณภาพและข้อผิดพลาด ในด้านคุณภาพและความถูกต้องของซอฟต์แวร์ ผู้วิจัยได้ประเมินจากจำนวนข้อผิดพลาด (Defects) และจำนวนรอบการแก้ไข (Rework) ที่เกิดขึ้นจริงในระหว่างกระบวนการพัฒนาจนแอปพลิเคชันเสร็จสมบูรณ์ โดยสรุปผลได้ดังแสดงในตารางที่ 2

ตารางที่ 2 สรุปจำนวนข้อผิดพลาดและรอบการแก้ไข

Table 2. Summary of Defects and Rework Cycles

กลุ่มการพัฒนา (Development Group)	**จำนวนข้อผิดพลาด** (Total Defects)	**ข้อผิดพลาดร้ายแรง** (High Severity)	**รอบการแก้ไข** (Total Rework)
การพัฒนาด้วยเอไอ (AI-Development)	6	3	9
การพัฒนาโดยมนุษย์ (Human-Development)	3	2	4

ผลการทดสอบชี้ให้เห็นว่าแม้ AI จะมีความเร็วสูง แต่กลับพบข้อผิดพลาดมากกว่ามนุษย์ถึง 2 เท่า โดยเฉพาะปัญหาเรื่องความเข้ากันได้ของเวอร์ชันไลบรารี (Library Mismatch) และตรรกะการคำนวณที่ผิดพลาดในโหมด Tournament ซึ่งสะท้อนให้เห็นถึงข้อจำกัดของ AI ในด้านความตระหนักรู้ต่อบริบททางเทคนิคที่เป็นปัจจุบัน ทั้งนี้ แอปพลิเคชันจากทั้งสองกลุ่มสามารถผ่านการทดสอบตามมาตรฐาน ISO/IEC 25010 ในด้านฟังก์ชันการทำงาน ความปลอดภัย และประสิทธิภาพการทำงานได้ครบถ้วนเมื่อสิ้นสุดกระบวนการพัฒนา

อภิปรายผล

ปัญญาประดิษฐ์สามารถเขียนโค้ดได้รวดเร็วและมีคุณภาพตามคำสั่ง (Prompt) ที่ได้รับ อย่างไรก็ตาม ยังคงมีจุดอ่อนในเรื่องของการสร้างโค้ดส่วนเกินที่ไม่ได้ใช้งานจริง รวมถึงข้อจำกัดด้านความรู้ที่เป็นปัจจุบันและบริบทเฉพาะของเครื่องมือ เช่น การแนะนำไลบรารีที่มีปัญหาความไม่เข้ากันกับเวอร์ชันปัจจุบันของ Expo Go ซึ่งสะท้อนให้เห็นว่าปัญญาประดิษฐ์ไม่ได้ทำการทดสอบด้วยโค้ดจริง แต่เป็นการคาดเดาจากรูปแบบข้อมูลที่มีอยู่ ทำให้มนุษย์ยังคงต้องเข้ามาจัดการปัญหาทางเทคนิคที่ซับซ้อน

ผลการทดลองยืนยันว่าแนวคิดการทำงานร่วมกันระหว่างมนุษย์และปัญญาประดิษฐ์ (Human-in-the-Loop) มีความสำคัญอย่างยิ่ง แม้ว่าในช่วงการพัฒนาปัญญาประดิษฐ์จะทำงานได้เร็วกว่ามนุษย์ถึง 5 เท่า แต่ในขั้นตอนการแก้ไขและบำรุงรักษา ทั้งมนุษย์และปัญญาประดิษฐ์กลับใช้เวลาใกล้เคียงกันในการไล่หาจุดผิดพลาดและแก้ไข หากขาดการตรวจสอบจากมนุษย์ที่มีความรู้ แอปพลิเคชันที่พัฒนาด้วยปัญญาประดิษฐ์อาจไม่สามารถใช้งานได้จริงเนื่องจากข้อผิดพลาดที่ร้ายแรง นอกจากนี้ ปัญญาประดิษฐ์ยังมีประสิทธิภาพสูงในงานที่ทำซ้ำและมีรูปแบบชัดเจน แต่จะเริ่มแสดงข้อผิดพลาดทางตรรกะเมื่อต้องเผชิญกับฟังก์ชันที่มีความซับซ้อนเฉพาะทาง

สรุปผลข้อเสนอแนะ

การศึกษาพบว่าการใช้ปัญญาประดิษฐ์มีความโดดเด่นอย่างมากในด้านประสิทธิภาพการผลิต โดยสามารถลดระยะเวลาการพัฒนาโดยรวมเหลือเพียง 88 นาที เมื่อเทียบกับมนุษย์ที่ใช้เวลา 295 นาที โดยเฉพาะในช่วงการขึ้นโครงร่างโปรแกรมที่ปัญญาประดิษฐ์ทำได้เร็วกว่ามนุษย์อย่างเห็นได้ชัดเนื่องจากสามารถสร้างโค้ดจำนวนมากได้ทันที อย่างไรก็ตาม ปัญญาประดิษฐ์ยังคงมีสัดส่วนการใช้เวลาในการแก้บั๊กค่อนข้างสูงเมื่อเทียบกับเวลาที่ใช้สร้าง ในด้านคุณภาพซอฟต์แวร์ตามมาตรฐาน ISO/IEC 25010 แอปพลิเคชันทั้งสองรูปแบบมีคุณภาพใกล้เคียงกันเมื่อเสร็จสมบูรณ์ แม้ปัญญาประดิษฐ์จะพบข้อผิดพลาดมากกว่าและมีความรุนแรงสูงกว่าในบางกรณี แต่โครงสร้างโค้ดที่สร้างขึ้นมีความเป็นระเบียบและง่ายต่อการบำรุงรักษา

สำหรับการศึกษาในอนาคต ควรมีการเพิ่มความซับซ้อนของโจทย์เพื่อทดสอบขีดจำกัดของปัญญาประดิษฐ์ในโครงการที่มีขนาดใหญ่ขึ้น และควรมีการทดสอบเปรียบเทียบกับปัญญาประดิษฐ์โมเดลอื่นๆ เพื่อวิเคราะห์เครื่องมือที่มีประสิทธิภาพสูงสุดสำหรับการพัฒนาซอฟต์แวร์

การนำไปใช้

ควรใช้ปัญญาประดิษฐ์ในฐานะผู้ช่วยงานไม่ใช่ผู้ทดแทน โดยเหมาะสำหรับการทำงานในขั้นต้น เช่น การสร้างฟังก์ชันพื้นฐาน การตกแต่งส่วนติดต่อผู้ใช้งานเบื้องต้น หรือการเสนอแนวคิดในช่วงเริ่มต้นโครงการเพื่อลดระยะเวลาการทำงานที่ซ้ำซ้อน สิ่งสำคัญคือนักพัฒนาไม่ควรคาดหวังให้ปัญญาประดิษฐ์ทำงานแทนจนจบกระบวนการโดยไม่มีการตรวจสอบ ในการใช้งานจริงควรระบุเวอร์ชันของเครื่องมือที่ใช้พัฒนาให้ชัดเจนในคำสั่ง (Prompt) ตั้งแต่แรกเพื่อลดปัญหาความไม่เข้ากันของเครื่องมือ ทั้งนี้นักพัฒนา

ซอฟต์แวร์ยังคงจำเป็นต้องมีความรู้พื้นฐานที่แข็งแกร่งในการอ่านโค้ดและแก้ปัญหา เพื่อให้สามารถจัดการกับข้อผิดพลาดที่ปัญญาประดิษฐ์สร้างขึ้นได้อย่างรวดเร็วและมีประสิทธิภาพ

กิตติกรรมประกาศ

รายงานการวิจัยฉบับนี้สำเร็จลุล่วงไปได้ด้วยดีเนื่องจากได้รับความเมตตาและความกรุณาอย่างสูงจาก ผู้ช่วยศาสตราจารย์ ดร.ศุภกฤษ นิวัฒนากูล อาจารย์ที่ปรึกษาโครงงาน ที่ได้สละเวลาให้คำปรึกษา แนะนำแนวทางที่ถูกต้อง และช่วยตรวจทานแก้ไขข้อบกพร่องต่าง ๆ ของงานวิจัยด้วยความเอาใจใส่เสมอมา จนทำให้งานวิจัยมีความสมบูรณ์และถูกต้องตามหลักวิชาการ ผู้วิจัยขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

งานวิจัยฉบับนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาการสารสนเทศบัณฑิต ในรายวิชาโครงงานเทคโนโลยีดิจิทัล สาขาวิชาเทคโนโลยีดิจิทัล สำนักวิชาศาสตร์และศิลป์ดิจิทัล มหาวิทยาลัยเทคโนโลยีสุรนารี ผู้วิจัยขอขอบพระคุณคณาจารย์ทุกท่านที่ได้ประสิทธิ์ประสาทวิชาความรู้และทักษะที่เป็นประโยชน์ ตลอดจนขอขอบคุณเพื่อนนักศึกษาและผู้ที่มีส่วนเกี่ยวข้องทุกท่านที่ได้ให้ความช่วยเหลือในการรวบรวมข้อมูลและเตรียมเอกสารรายงานจนสำเร็จเสร็จสิ้นตามวัตถุประสงค์

ท้ายที่สุดนี้ ผู้วิจัยหวังว่าคุณค่าและประโยชน์อันเกิดจากงานวิจัยเรื่องการเปรียบเทียบคุณภาพและประสิทธิภาพระหว่างแอปพลิเคชันที่พัฒนาด้วยปัญญาประดิษฐ์กับแอปพลิเคชันที่พัฒนาโดยมนุษย์ฉบับนี้ จะเป็นประโยชน์ต่อวงการวิชาการและผู้ที่สนใจนำปัญญาประดิษฐ์เชิงสร้างสรรค์มาประยุกต์ใช้ในการพัฒนาซอฟต์แวร์ต่อไปในอนาคต

เอกสารอ้างอิง

Surapapha, P., & Paireekreng, W. (2024). A comparative study of efficiency and quality of software project deliverables in Thai government agencies: ISO/IEC 29110 vs. 3-phase quality inspection with ISO/IEC 29110 [In Thai]. Journal of Business Administration, 14(2), 65–82.

Sauvola, J., Tarkoma, S., Klemettinen, M., Riekki, J., & Doermann, D. S. (2024). Future of software development with generative AI. Automated Software Engineering, 31(1), Article 26. https://doi.org/10.1007/s10515-024-00426-z

Farias, J. P., et al. (2021). Tool for Measuring Productivity in Software Development Teams. Information, 12(10), 396. https://doi.org/10.3390/info12100396